



Interrogación 1 – Pauta

1. Respuesta:

- a) La gramática es ambigua porque la sentencia

$$\text{if } (E_1) \text{ if } (E_2) S_1 \text{ else } S_2$$

admite dos árboles de derivación distintos.

Derivación 1: el **else** se asocia al *if* interior.

$$\begin{aligned} \langle \text{stmt} \rangle &\Rightarrow \langle \text{if_stmt} \rangle \\ &\Rightarrow \text{if } (E_1) \langle \text{stmt} \rangle \\ &\Rightarrow \text{if } (E_1) \langle \text{if_stmt} \rangle \\ &\Rightarrow \text{if } (E_1) \text{ if } (E_2) \langle \text{stmt} \rangle \text{ else } \langle \text{stmt} \rangle \\ &\Rightarrow \text{if } (E_1) \text{ if } (E_2) S_1 \text{ else } S_2 \end{aligned}$$

En este árbol, el **else** pertenece al *if* interior. Si E_1 es falso, no se ejecuta nada.

Derivación 2: el **else** se asocia al *if* exterior.

$$\begin{aligned} \langle \text{stmt} \rangle &\Rightarrow \langle \text{if_stmt} \rangle \\ &\Rightarrow \text{if } (E_1) \langle \text{stmt} \rangle \text{ else } \langle \text{stmt} \rangle \\ &\Rightarrow \text{if } (E_1) \langle \text{if_stmt} \rangle \text{ else } \langle \text{stmt} \rangle \\ &\Rightarrow \text{if } (E_1) \text{ if } (E_2) \langle \text{stmt} \rangle \text{ else } \langle \text{stmt} \rangle \\ &\Rightarrow \text{if } (E_1) \text{ if } (E_2) S_1 \text{ else } S_2 \end{aligned}$$

En este árbol, el **else** pertenece al *if* exterior. Si E_1 es falso, se ejecuta S_2 .

Como la misma cadena tiene dos árboles de derivación distintos, la gramática es ambigua.

- b) La gramática no es ambigua porque los no terminales $\langle \text{matched} \rangle$ y $\langle \text{unmatched} \rangle$ particionan las sentencias de forma disjunta: $\langle \text{matched} \rangle$ solo puede derivar sentencias donde todo **if** tiene su **else**, y $\langle \text{unmatched} \rangle$ solo puede derivar sentencias donde algún **if** no lo tiene.

La regla clave es que en $\langle \text{matched} \rangle$, la rama *then* debe ser también $\langle \text{matched} \rangle$. Esto impide que un **else** se asocie a un **if** exterior, ya que si la rama *then* contiene un **if** sin **else**, esa rama es $\langle \text{unmatched} \rangle$ y no puede aparecer dentro de $\langle \text{matched} \rangle$.

Por lo tanto, todo **else** queda forzosamente asociado al **if** abierto más cercano, eliminando la ambigüedad del *dangling else*.

Puntaje:

- (a) (1 pts) Por obtener un árbol de derivación.
- (a) (2 pts) Por derivar un segundo árbol de derivación, distinto del primero.
- (b) (1 pts) Dar a entender que se debe a la falta de ambigüedad al asignar if-else.
- (b) (2 pts) Por justificar correctamente el funcionamiento de *match* y *unmatched*

2. **Respuesta:** El stack de registros de activación después de ejecutar `fie(2)` es el siguiente:

Frame	Variable	Tipo	Valor
A	y	int	0
B	x	int	0
	fie	fun	—
C	x	int	1
fie	n	int	2

Cadena estática: cada registro apunta al registro del bloque que lo *contiene léxicamente*. `fie` está definida en B, por lo que su cadena estática apunta al registro de B. C está dentro de B, por lo que su cadena estática apunta a B. B está dentro de A, por lo que su cadena estática apunta a A.

- $\text{fie} \rightarrow \text{B} \rightarrow \text{A}$
- $\text{C} \rightarrow \text{B} \rightarrow \text{A}$

Cadena dinámica: cada registro apunta al registro del bloque que lo *invocó en tiempo de ejecución*. El orden de activación es $\text{A} \rightarrow \text{B} \rightarrow \text{C} \rightarrow \text{fie}$, por lo que cada registro apunta al inmediatamente anterior en esa secuencia.

- $\text{fie} \rightarrow \text{C} \rightarrow \text{B} \rightarrow \text{A}$

Puntaje:

- (4 pts) Por el stack.
- (1 pto) Por las cadenas estáticas.
- (1 pto) Por la cadena dinámica.

3. Respuesta:

- a) **Sí:** En una función tail recursive, la llamada recursiva es la absolutamente última operación ejecutada, lo que permite que el programa reuse el stack frame vigente. En este caso, ya sea que foo ejecute **return foo(a-b, b)** o **return foo(a, b-a)**, se cumple la condición anterior.
- b) **No:** En una función tail recursive, la llamada recursiva es la absolutamente última operación ejecutada, lo que permite que el programa reuse el stack frame vigente. Pero esta función foo contiene una **llamada recursiva anidada**. El programa debe calcular el resultado de la llamada “interior” foo(x+11) primero, y luego ejecutar una segunda llamada recursiva “exterior” sobre el resultado anterior antes de terminar. Como la llamada exterior está esperando a que la llamada interior se resuelva, requiere stack frames activos.

Puntaje:

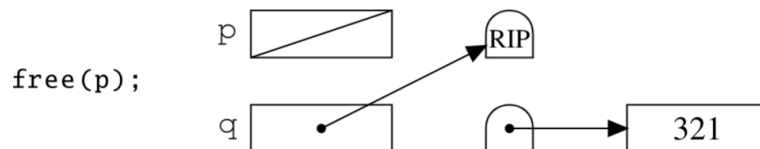
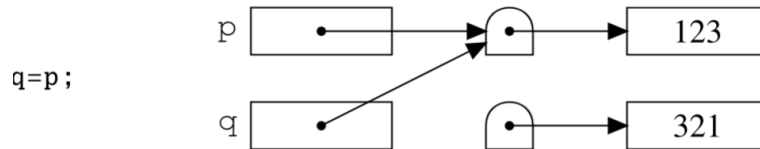
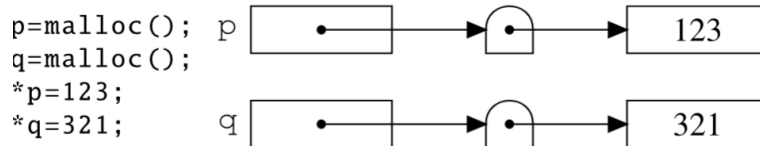
- (a) (1 pto) Por Respuesta correcta
- (a) (2 pts) Por justificar correctamente en base al return del programa.
- (b) (1 pto) Por Respuesta correcta
- (b) (2 pts) Por justificar correctamente en base al return del programa.

4. Respuesta:

a) Lápidas

Cada objeto en el heap tiene asociada una *lápida*: una celda intermedia que contiene la dirección real del objeto. Todos los punteros al objeto apuntan a la lápida, no directamente al objeto. Al liberar el objeto con `free(p)`, la lápida se invalida (se marca con un valor especial). Cualquier intento posterior de desreferenciar un puntero que apunta a esa lápida es detectado como acceso inválido.

Evolución del estado con lápidas para el código dado:

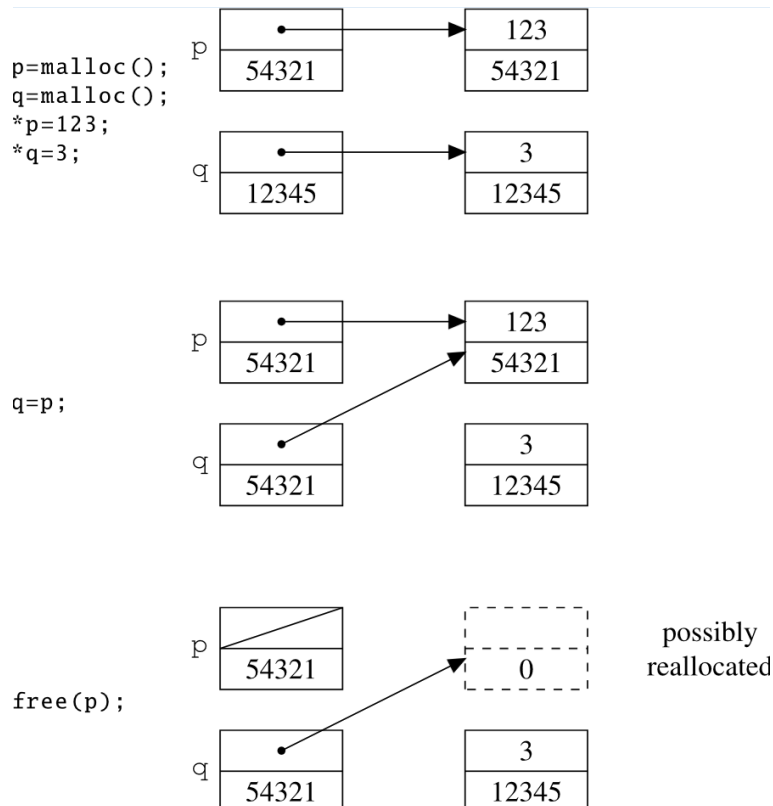


b) Candados y llaves

A cada objeto en el heap se le asigna un entero único (*candado*) que se almacena junto a él. Cada puntero lleva consigo una copia de ese entero (*llave*). Al desreferenciar un puntero se comprueba que su llave coincida con el candado del objeto; si no coinciden, el acceso es inválido. Al liberar un objeto, su candado se borra (o se cambia), de modo que futuras comparaciones fallen.

Los valores enteros usados para los candados son 54321 (para `malloc()` de `p`) y 12345 (para `malloc()` de `q`).

Evolución del estado con candados y llaves:



Puntaje:

- (a) (1 pto) Representación correcta de elementos
- (a) (1 pto) Manejo correcto de referencias y asignaciones.
- (a) (1 pto) Funcionamiento de `free(p)`.
- (b) (1 pto) Representación correcta de elementos
- (b) (1 pto) Manejo correcto de referencias y asignaciones.
- (b) (1 pto) Funcionamiento de `free(p)`.

5. **Respuesta:**

La jerarquía de clases es:

$$A \leftarrow B \leftarrow C \leftarrow D$$

donde **A** y **B** son abstractas. La clase **C** define **foo** con **return x+val+n**, y **D** redefine **foo** con la misma expresión **return x+val+n** pero con su propio **n** (inicializado en el constructor).

Al ejecutar **D d = new D(3)**, el constructor **D(int v){n=v;}** fija **d.n = 3**.

Las asignaciones **a = d**, **b = d**, **c = d** hacen que **a**, **b**, **c** apunten al mismo objeto de tipo **D**. Por despacho dinámico, todas las llamadas **foo(1)** invocan el método de **D**.

El método **foo** de **D** retorna $x + \text{val} + n$, donde:

- $x = 1$ (argumento).
- **val**: se hereda de la clase más cercana en la jerarquía que lo defina. **D** hereda de **C**, que hereda de **B** donde **val = 2** (redefine el **val = 1** de **A**). Por lo tanto **val = 2**.
- $n = 3$ (asignado en el constructor).

Resultado: $\text{foo}(1) = 1 + 2 + 3 = 6$.

$$u = 6, \quad v = 6, \quad w = 6, \quad z = 6$$

Las cuatro variables valen lo mismo porque **a**, **b**, **c**, **d** son todas referencias al mismo objeto de clase **D**, y el despacho dinámico siempre selecciona el método **foo** de **D**.

Puntaje:

- **(1 pto)** Por variable correcta (4).
- **(2 pts)** Por el valor de todas las variables.